

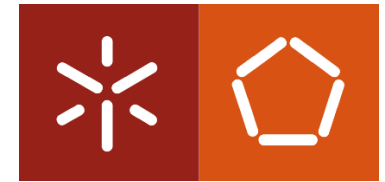
Algoritmos e Protocolos de Encaminhamento (parte 2)

**Licenciatura em Engenharia de
Telecomunicações e Informática**

3º ano - 2º Semestre

2024/2025





Encaminhamento na Internet

- **Reenvio (forwarding) num encaminhador:**

- Comuta os pacotes que recebe através de uma interface de rede (de entrada) para uma interface de rede (de saída) mediante a informação que encontra na tabela de encaminhamento: **Plano de DADOS**

- **Encaminhamento (routing):**

- Determina a(s) melhor(es) rotas que os pacotes devem seguir desde o nó fonte até ao nó destino: : **Plano de CONTROLO**

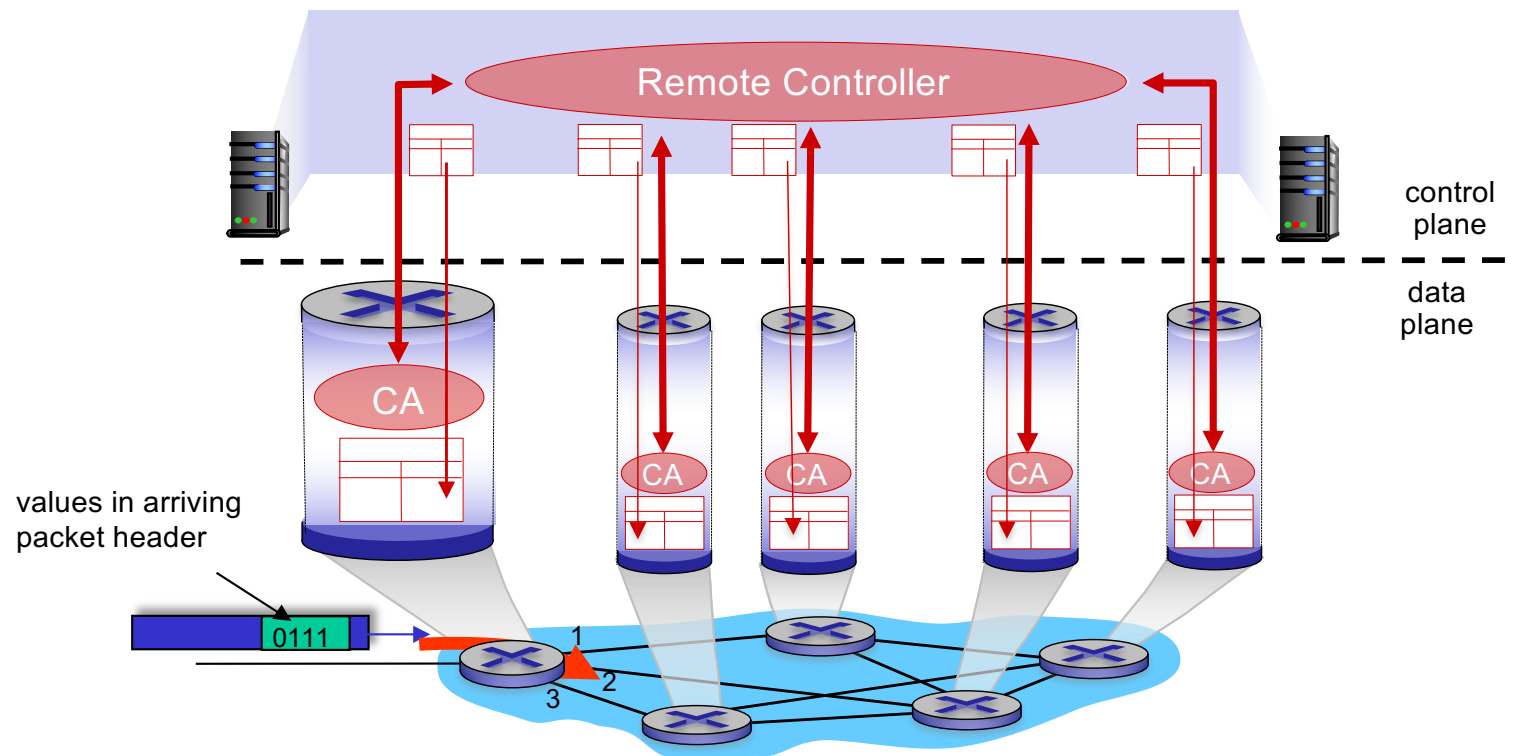
- Atualmente há duas alternativas para estruturar o plano de controlo:

- Encaminhamento distribuído implementado por cada encaminhador (router): abordagem tradicional;
- Encaminhamento centralizado (redes definidas por software);

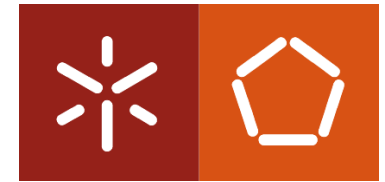


Encaminhamento centralizado (SDNs)

- O controlador central (Remote Control) calcula os melhores caminhos para todos os destinos possíveis e instala as as entradas correspondentes nas tabelas de encaminhamento usando um agente local (Control Agent).

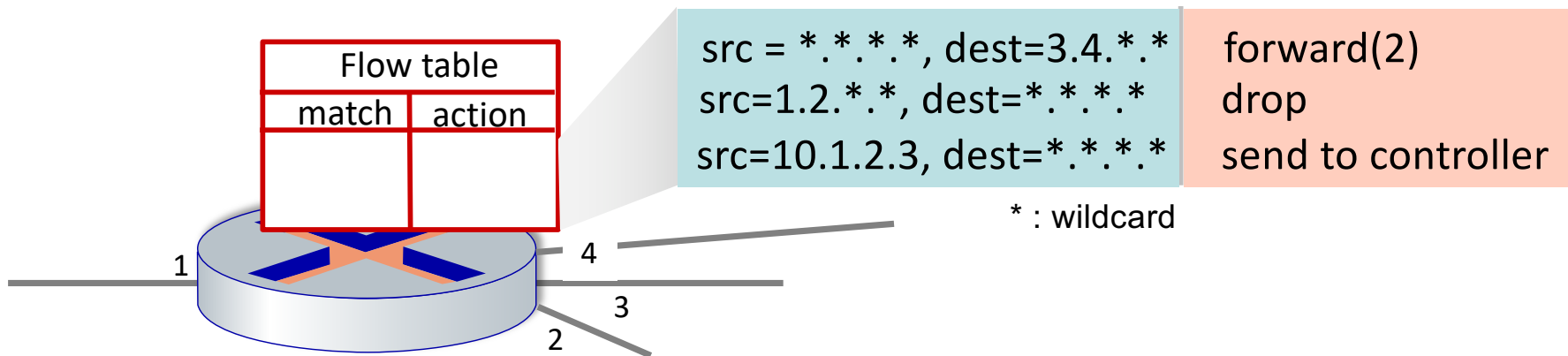


Tabelas de Fluxos

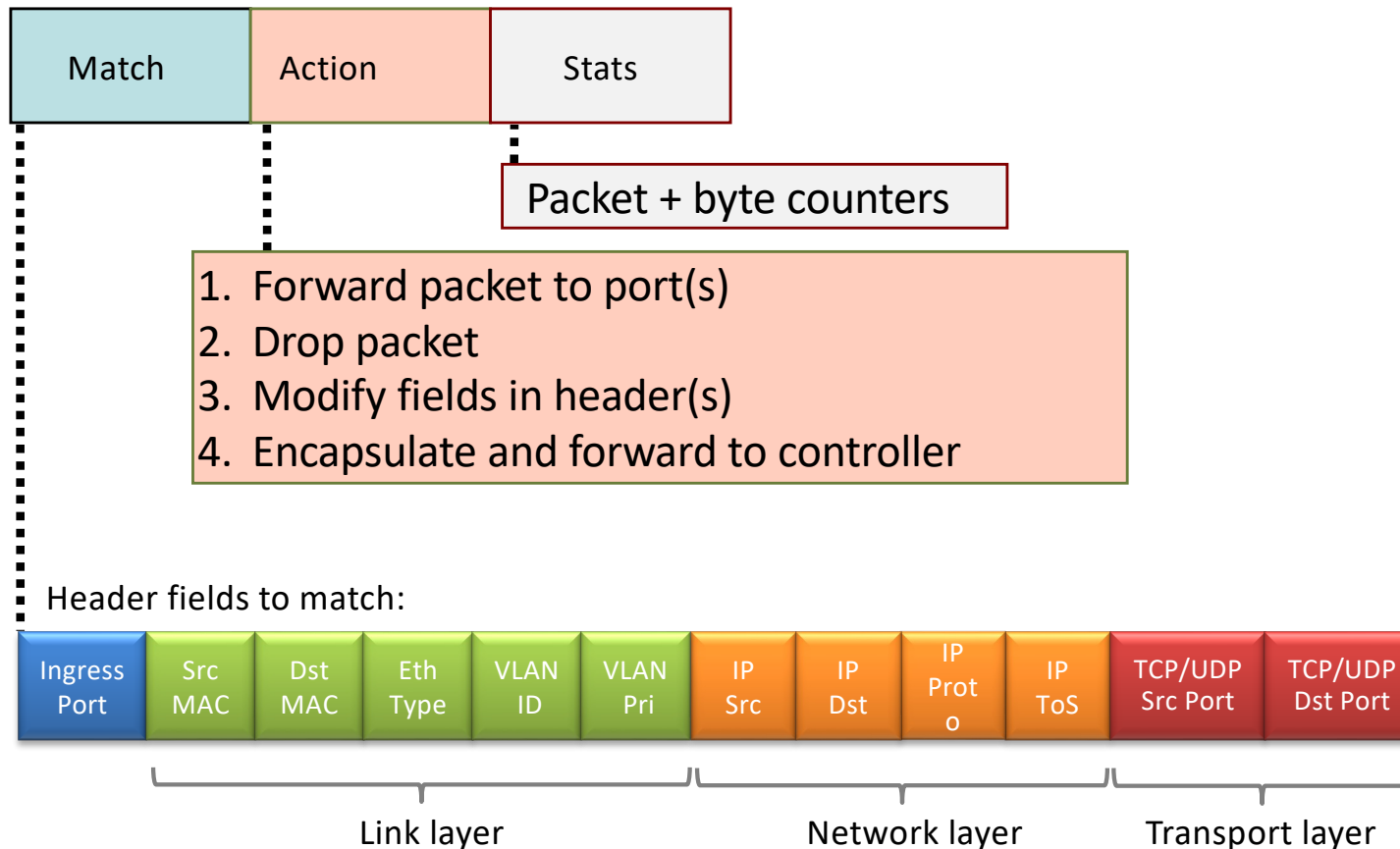


● Generalização do forwarding (re-envio) dos pacotes com base no endereço destino

- Os diferentes campos dos cabeçalhos dos pacotes podem ser usados como chave de pesquisa nas tabelas (tabelas de fluxos/flow tables)
- E podem desencadear diferentes tipos de ações: re-envia, descarta, modifica, etc



OpenFlow: tabelas de fluxos



OpenFlow: exemplos



Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	*	22	drop

Block (do not forward) all datagrams destined to TCP port 22 (ssh port #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

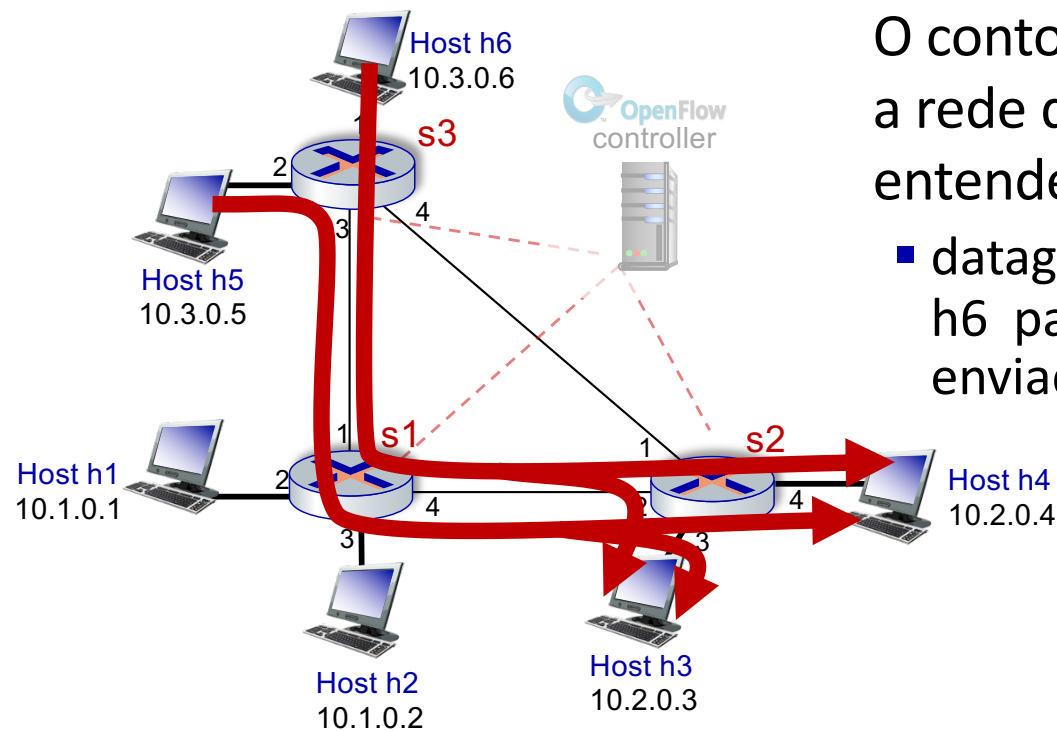
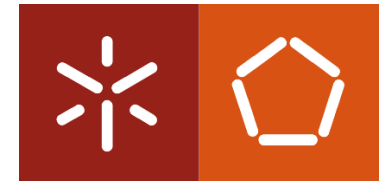
Block (do not forward) all datagrams sent by host 128.119.1.1

Layer 2 destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

layer 2 frames with destination MAC address 22:A7:23:11:E1:02 should be forwarded to output port 3

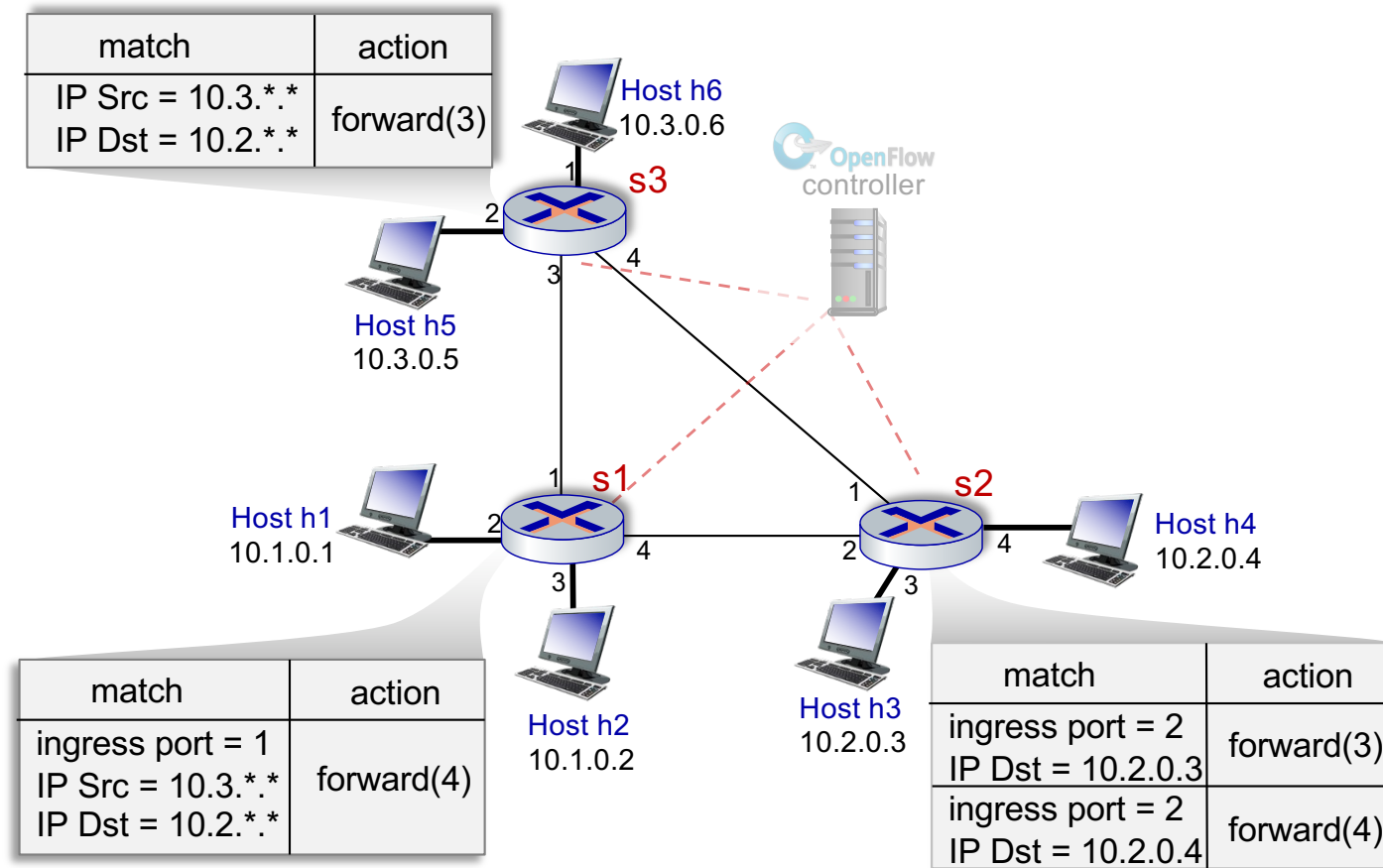
OpenFlow: mais um exemplo



O controlador pode programar a rede da forma que entender, por exemplo:

- datagramas enviados por h5 e h6 para h3 or h4 devem ser enviados através de s1 e s2.

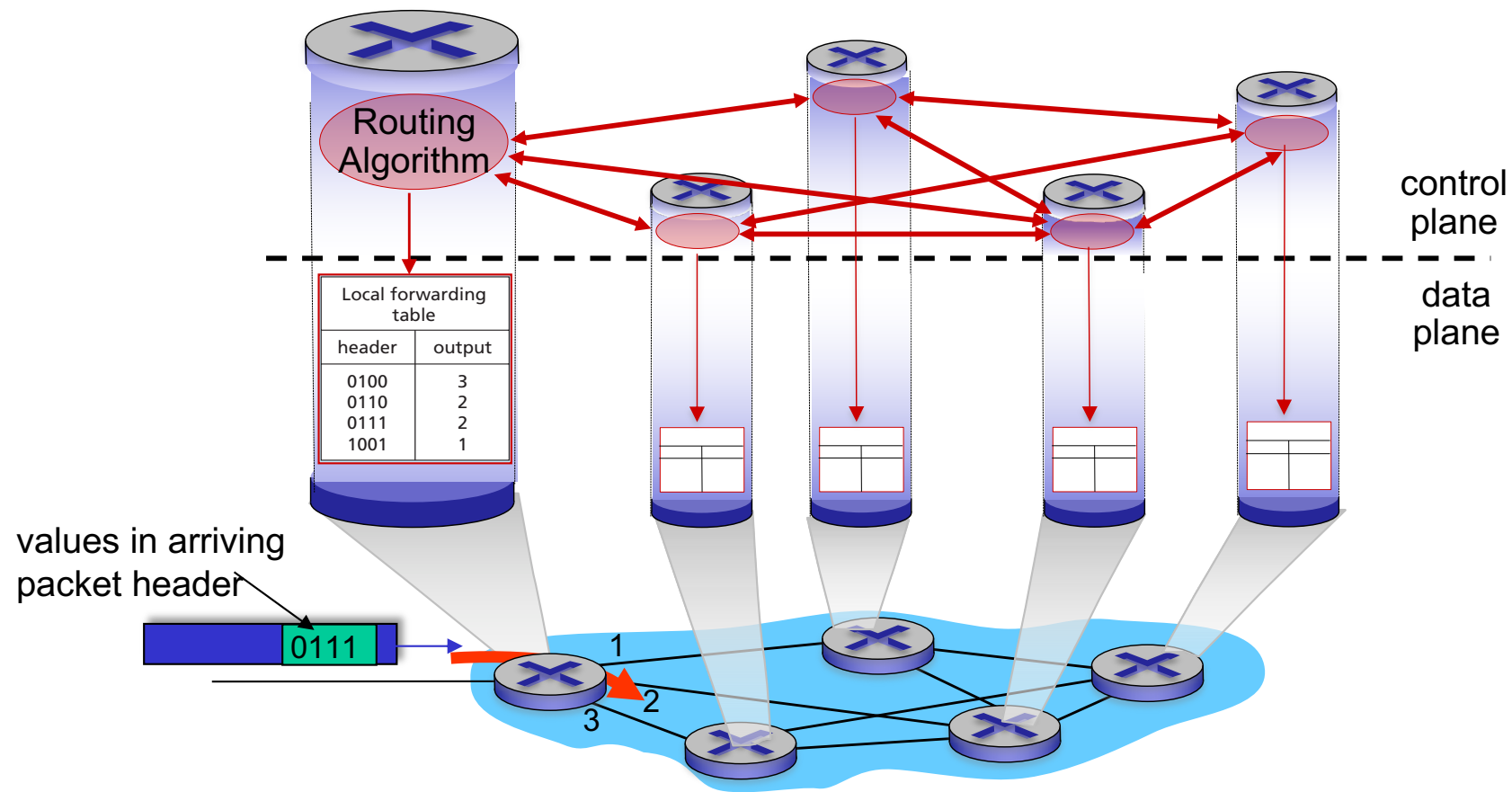
OpenFlow: mais um exemplo





Encaminhamento distribuído (tradicional)

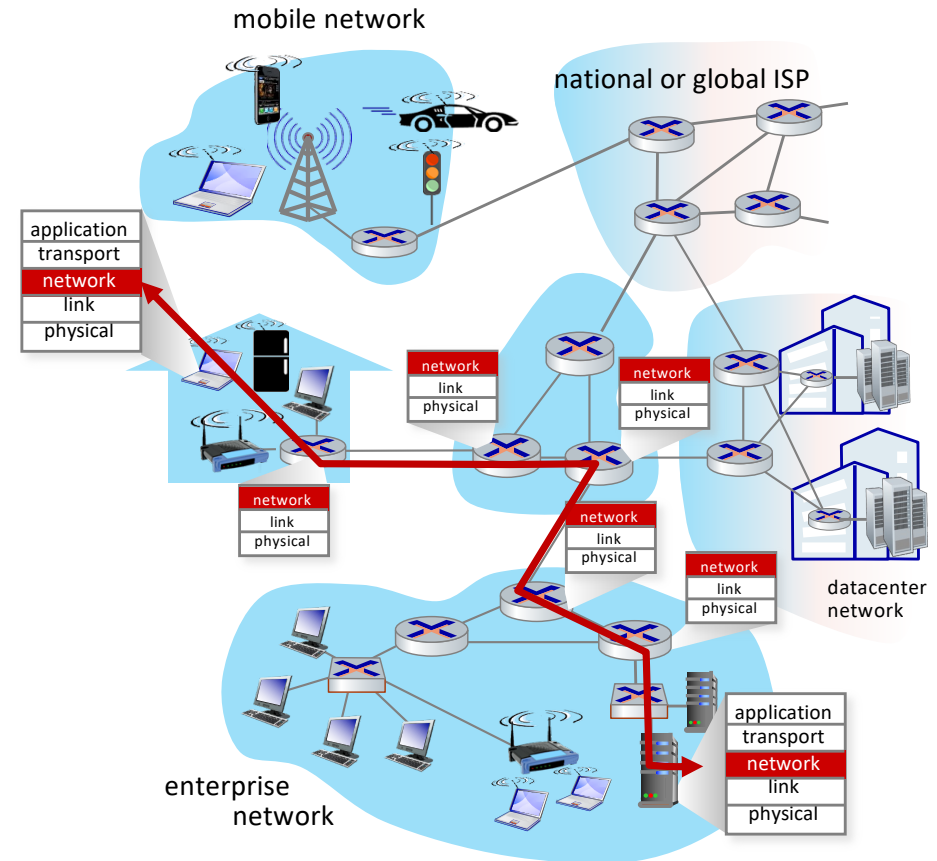
- No plano de controlo existem componentes individuais em cada um dos routers que interagem para encontrar os melhores caminhos para cada destino possível.



Algoritmos de Encaminhamento



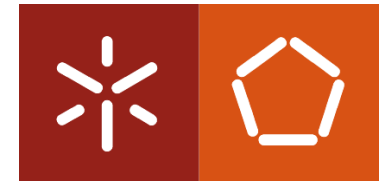
- **Objetivo: determinar a(s) melhor(es) rotas que os pacotes devem seguir desde o nó fonte até ao nó destino, através de uma rede de routers**
 - “rota”: sequência de routers que os pacotes devem atravessar desde o nó fonte até ao nó destino;
 - “melhor”: menor “custo”, “mais rápida”, “menos congestionada”;
 - O encaminhamento é um dos maiores desafios na área das redes.





● Grafo

- Um **grafo** $G=(V,A)$ é constituído por um conjunto de V **vértices** e por uma colecção de A **arestas**, onde cada aresta pertencente a A corresponde a um par de vértices pertencente a V .
- Cada aresta tem associado um valor que representa o seu **custo**.
- Um grafo **não dirigido** é um grafo onde a aresta que une os vértices x e y tem o mesmo custo que a aresta que une os vértices y e x . Um grafo **dirigido** pelo contrário é um grafo onde duas arestas que unem dois quaisquer vértices (x e y) têm custos distintos (de x para y e de y para x)
- Um vértice x é **vizinho** de um vértice y se existe uma aresta entre x e y .



Encaminhamento em redes IP

Abstração

● A topologia de rede é um Grafo:

- Os nós do grafo são os encaminhadores
- Os arcos do grafo são as ligações da rede
- O custo das ligações pode ser estabelecido em função do atraso, da capacidade, do nível congestão, do custo, da distância, etc
- Um “bom” caminho significa tipicamente o caminho de “custo mínimo”, mas há outras possibilidades...

Algoritmo Encaminhamento

Objetivo: dado um conjunto de encaminhadores com ligações de rede a interligá-los o objetivo do algoritmo de encaminhamento é determinar um “bom” caminho desde a fonte até ao destino



Modelo Abstracto de Rede

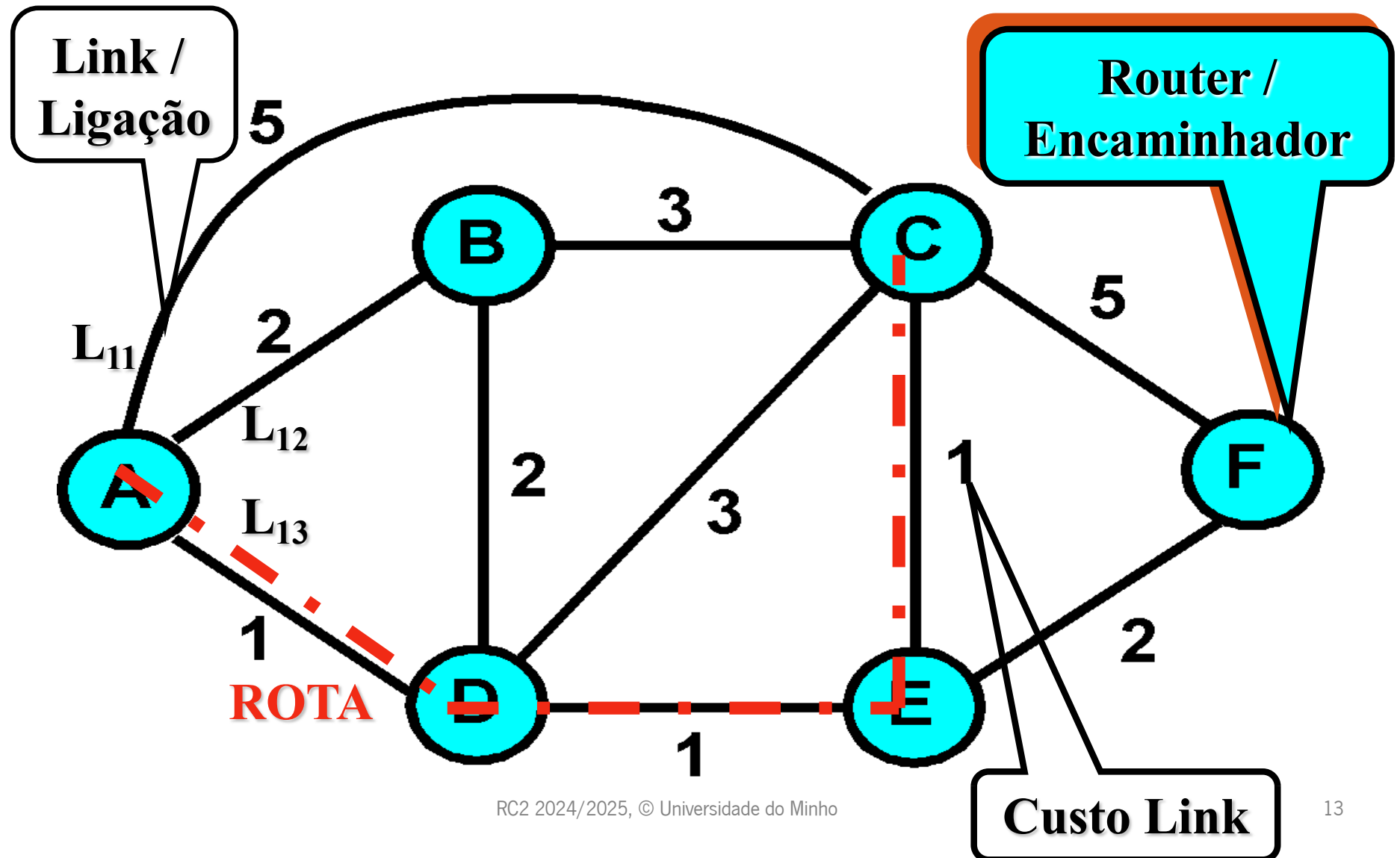




Tabela de Encaminhamento

NÓ A

Destino	Próximo Nó	Link	Custo
A	A	—	0
...	...	...	...
C	D	L ₁₃	3
...	...	...	...
F	D	L ₁₃	4



Estático ou dinâmico?

Estático:

- As rotas não mudam c/ tempo

Dinâmico:

- As rotas mudam c/ tempo
 - Actualização periódica
 - Em resposta à variação do custo das ligações

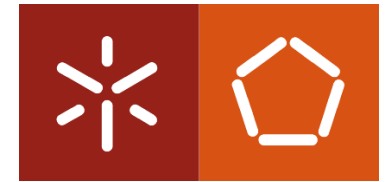
Informação Global ou Descentralizada?

Global:

- Todos os encaminhadores têm um conhecimento completo da topologia e custo das ligações
- Algoritmos de estado das ligações (LS-“link state”)

Descentralizada:

- Os encaminhadores só conhecem os vizinhos (fisicamente ligados) e o custo das ligações respectivas
- processo de computação é iterativo, troca de informação com os vizinhos
- Algoritmos de vector de distância (DV-“distance vector”)



Algoritmos do Estado das Ligações (LSA)

- **Todos os nós da topologia espalham pela rede o “estado das suas ligações” de forma a construírem a “base de dados topológica”**
 - Inicialmente necessitam de conhecer apenas os seus vizinhos directos, para quem enviam a identificação de todos os seus vizinhos bem como o custo das ligações que os separam deles
 - Um encaminhador ao receber esta informação actualiza a sua base de dados topológica e re-envia a informação para todos os seus vizinhos
 - Ao fim de algum tempo todos os nós possuem um conhecimento completo da topologia e dos custos de todas as ligações
- **Sobre esta informação, em cada encaminhador, é utilizado um algoritmo de descoberta dos caminhos de custo mínimo, tipicamente o algoritmo de Dijkstra.**
- **Com o resultado obtido da aplicação do algoritmo de Dijkstra, é preenchida a tabela de encaminhamento.**

Algoritmo de Dijkstra



- **Algoritmo Iterativo que ao fim de k iterações consegue descobrir os caminhos de custo mínimo de um determinado nó para k destinos**

- Seja $c(i,j)$ o custo da ligação do nó i para o nó j . Se o nó i e o nó j não estão directamente ligados $c(i,j)=\infty$;
- Seja $D(v)$ o custo do caminho desde o nó origem até ao nó v
- Seja $p(v)$ o nó que antecede v no caminho desde o nó origem até ao nó v
- Seja N o conjunto de todos os nós para os quais já se conhece o caminho de custo mínimo

Algoritmo de Dijkstra



1 *Initialization:*

2 $N' = \{u\}$ */* compute least cost path from u to all other nodes */*

3 *for all nodes v*

4 *if v adjacent to u* */* u initially knows direct-path-cost only to direct neighbors */*

5 *then $D(v) = c_{u,v}$* */* but may not be minimum cost! */*

6 *else $D(v) = \infty$*

7

8 *Loop*

9 *find w not in N' such that $D(w)$ is a minimum*

10 *add w to N'*

11 *update $D(v)$ for all v adjacent to w and not in N' :*

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

13 */* new least-path-cost to v is either old least-cost-path to v or known*

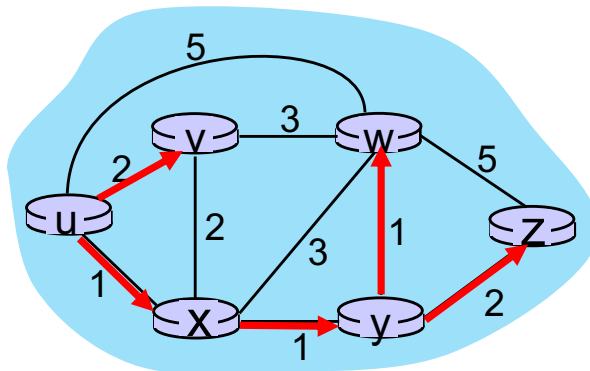
14 *least-cost-path to w plus direct-cost from w to v */*

15 *until all nodes in N'*

Algoritmo de Dijkstra (exemplo)



Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	u, x	2, u	4, x		2, x	∞
2	u, x, y	2, u	3, y			4, y
3	u, x, y, v		3, y			4, y
4	u, x, y, w					4, y
5	u, x, y, w, z					



find a not in N' such that $D(a)$ is a minimum

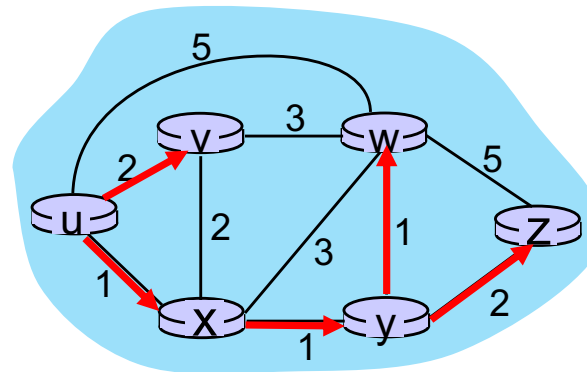
add a to N'

update $D(b)$ for all b adjacent to a and not in N' :

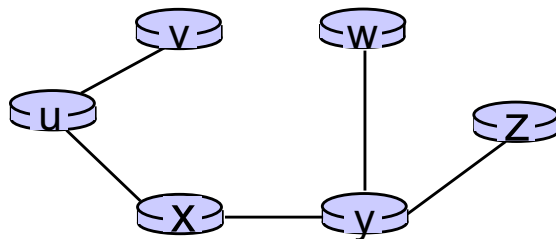
$$D(b) = \min (D(b), D(a) + c_{a,b})$$



Algoritmo de Dijkstra (exemplo)



Resulta na seguinte árvore de caminhos de custo mínimo:



Resulta na seguinte tabela de encaminhamento:

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

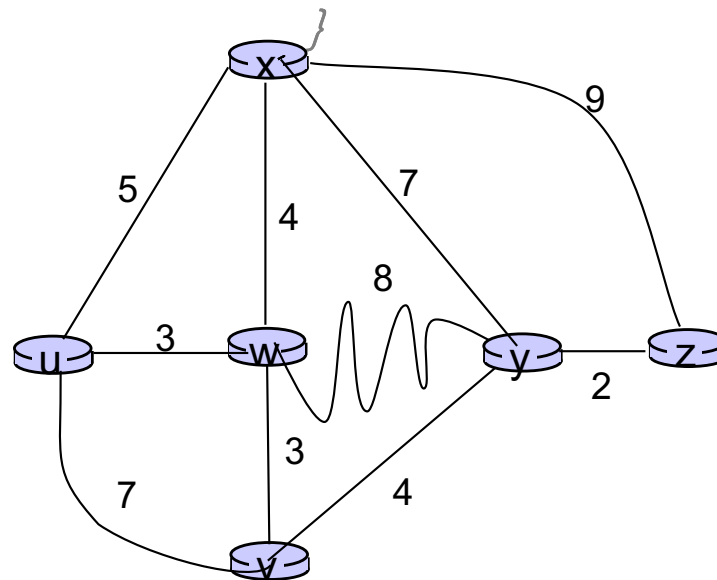
route from u to v directly
route from u to all other destinations via x

Algoritmo de Dijkstra (outro exemplo)



● **Exercício:**

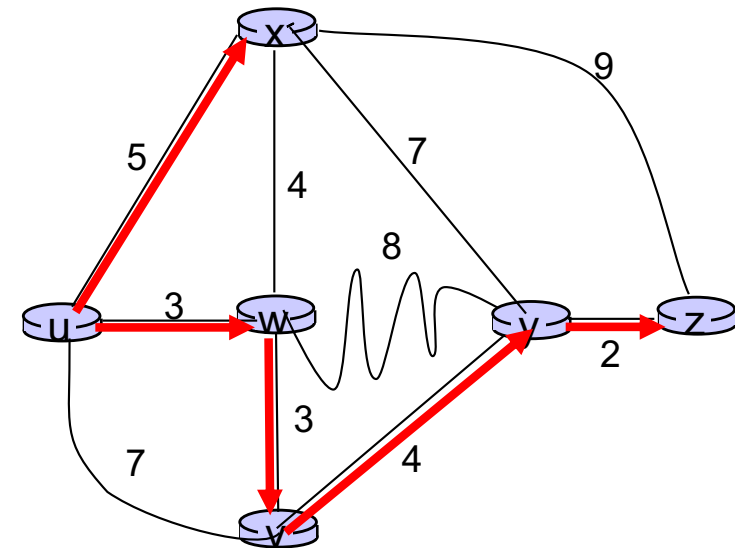
Utilizando o algoritmo de Dijkstra determine a tabela de encaminhamento do nó u, partindo do princípio que este nó tem um conhecimento completo da topologia da rede.



Algoritmo de Dijkstra (outro exemplo)



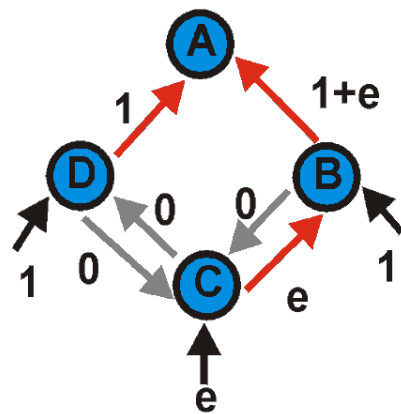
Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7, u	3, u	5, u	∞	∞
1	uw	6, w	5, u	11, w	∞	∞
2	uwx	6, w		11, w	14, x	
3	uwxv			10, v	14, x	
4	uwxvy				12, y	
5	uwxvyz					



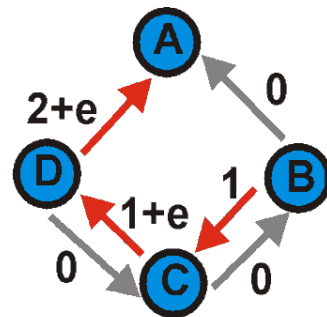
LSA – Oscilações



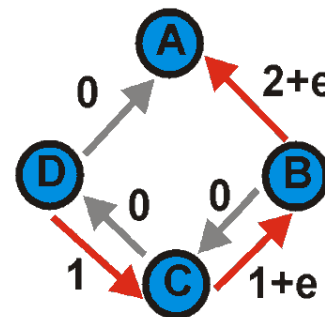
- **Este algoritmo pode, na presença de métricas que dependem do estado da rede, apresentar alguns problemas**
 - Por exemplo, se a métrica reflectir a carga nas ligações, sendo por isso uma métrica assimétrica
 - No exemplo, B e D enviam uma (1) unidade de tráfego para A e C envia e (<1) unidades de tráfego também para A



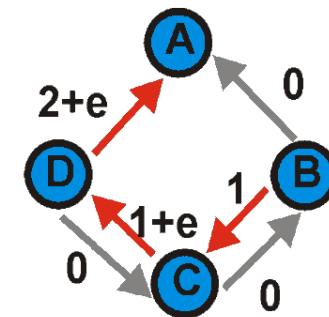
(a): initial routing



(b): B, C detect better path to A, clockwise



(c): B, C, D detect better path to A, counterclockwise



(d): B, C, D detect better path to A, clockwise

Algoritmo de Vectors de Distância (DVA)



- Ao contrário dos algoritmos de estado de ligação, os algoritmos de vectores de distância não usam informação global;
- São distribuídos, iterativos e assíncronos
 - Cada nó recebe informação de encaminhamento de algum dos seus vizinhos directos, recalcula a tabela de encaminhamento e envia essa informação de volta para os vizinhos;
 - O processo continua até que não haja informação de encaminhamento a ser trocada entre nós vizinhos
 - Não exige que os nós estejam sincronizados uns com os outros

Algoritmo de Vectores de Distância (DVA)



Baseado na equação de **Bellman-Ford** (BF) :

Esquação de Bellman-Ford

Seja $D_x(y)$: a distância do caminho de custo mínimo entre x e y
Então:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

Estimativa de v da distância do caminho de custo mínimo de v até y

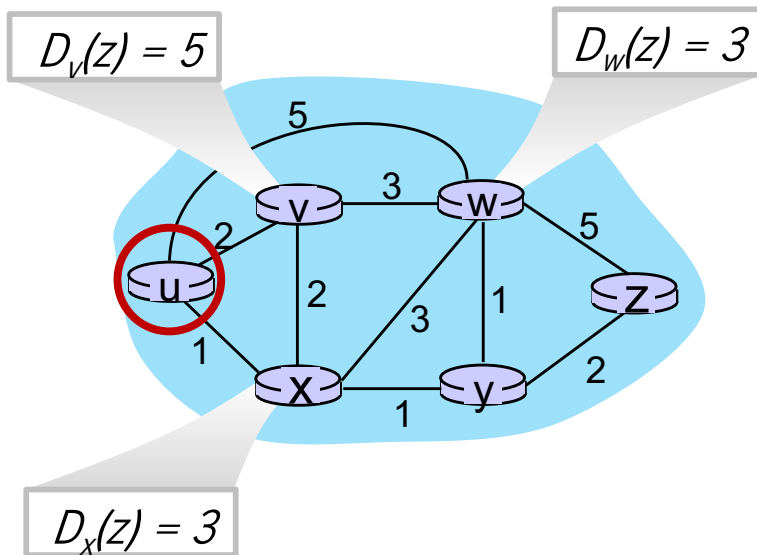
Custo da ligação directa entre x e v

Mínimo de todos os vizinhos v de x

Algoritmo de Vectors de Distâncias (exemplo)



Supondo que os vizinhos de u , x, v e w , conhecem a distância para chegar a z .



A equação de Bellman-Ford diz:

$$\begin{aligned} D_u(z) &= \min \{ c_{u,v} + D_v(z), \\ &\quad c_{u,x} + D_x(z), \\ &\quad c_{u,w} + D_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

O nó que consegue obter o menor valor é o próximo nó para chegar a z no caminho de custo mínimo para o destino z , ou seja o nó x seria o próximo nó

Algoritmo de Vectors de Distância (DVA)



Ideia chave:

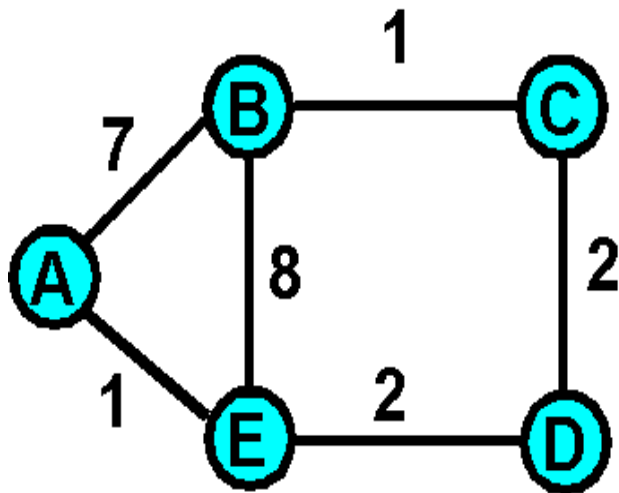
- *Periodicamente cada nó envia o seu vetor de distâncias aos seus vizinhos*
- *Quando x recebe um novo vetor de distâncias atualiza o seu próprio vetor usando a equação de Bellman-Ford. :*

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\} \text{ para cada nó } y \in N$$

- *A estimativa de $D_x(y)$ converge para o custo mínimo de x para y ($d_x(y)$)*



Tabela de distâncias



		cost to destination via		
d e s t i n a t.	$D^E()$	A	B	D
	A	1	14	5
	B	7	8	5
	C	6	9	4
	D	4	11	2

Da Tabela de Distâncias à Tabela de Encaminhamento



		Custo p/ o destino <u>via</u>		
destination	$D^E()$	A	B	D
	A	1	14	5
	B	7	8	5
	C	6	9	4
	D	4	11	2

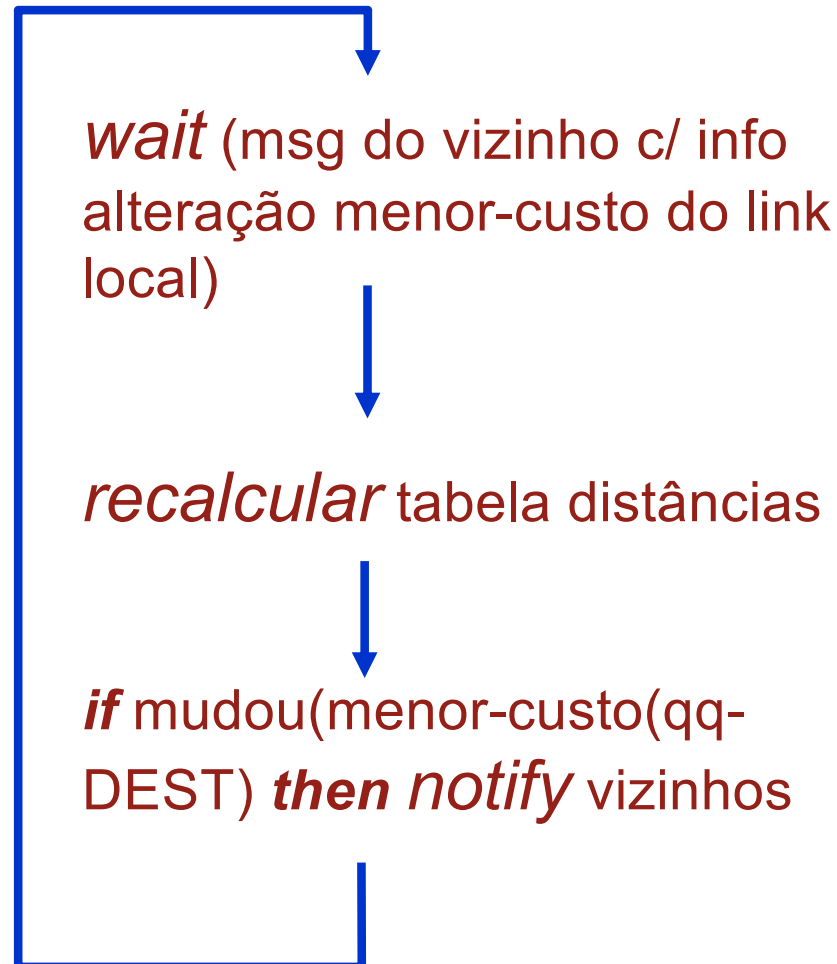
		Link Saída, custo	
destination			
	A	A,	1
	B	D,	5
	C	D,	4
	D	D,	2

Tabela Distâncias → Tabela de Routing



Algoritmos de Vector de Distâncias (DVA)

Cada nó:



Iterativo, assíncrono: cada iteração local é causada por:

- mudança custo link local
- mensagem do vizinho: vizinho anuncia novo custo

Distribuído:

- cada nó notifica vizinhos *só* quando muda o menor custo p/ qq destino
 - vizinhos notificam vizinhos (se necessário!)

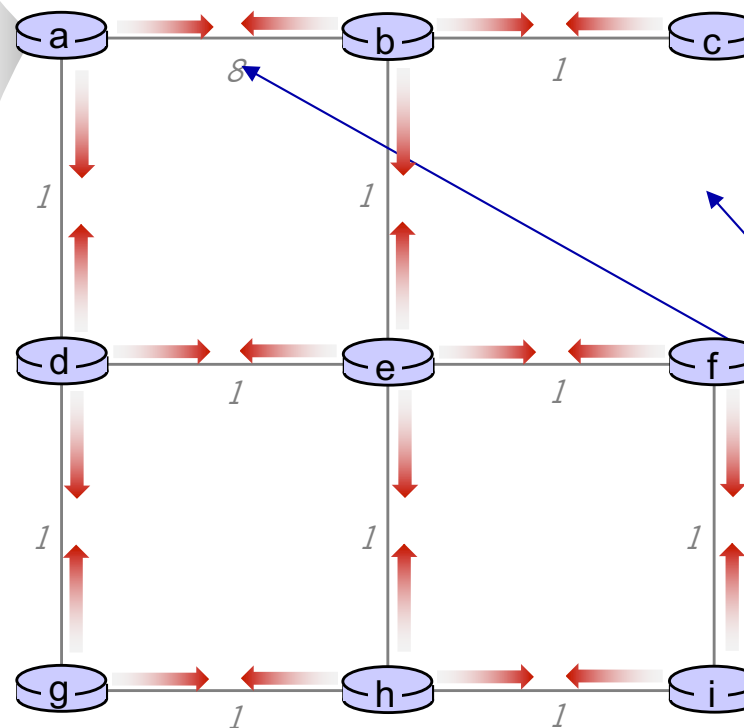
Algoritmo de Vetor de Distâncias (exemplo)



$t=0$

- All nodes have distance estimates to nearest neighbors (only)
- All nodes send their local distance vector to their neighbors

DV in a:	
$D_a(a)$	0
$D_a(b)$	8
$D_a(c)$	∞
$D_a(d)$	1
$D_a(e)$	∞
$D_a(f)$	∞
$D_a(g)$	∞
$D_a(h)$	∞
$D_a(i)$	∞



A few asymmetries:

- missing link
- larger cost

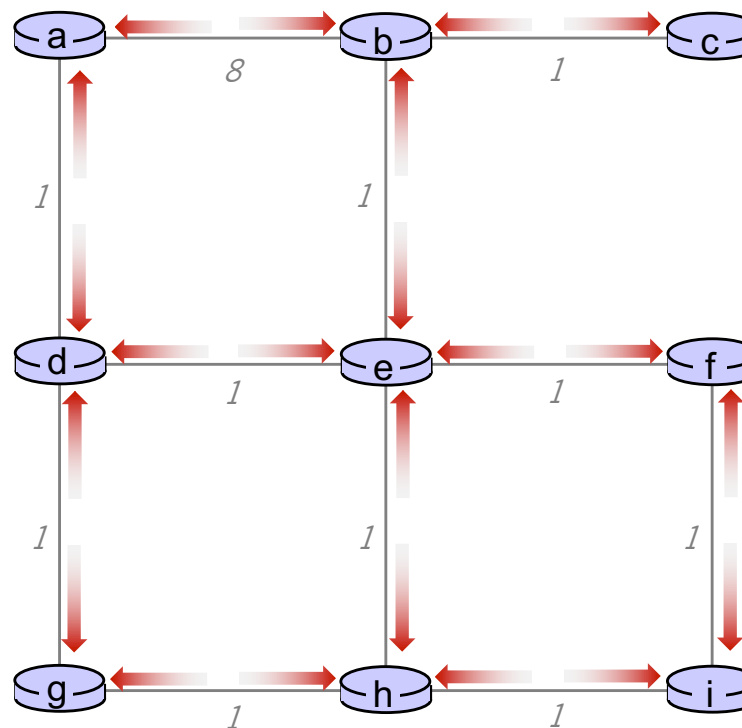
Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



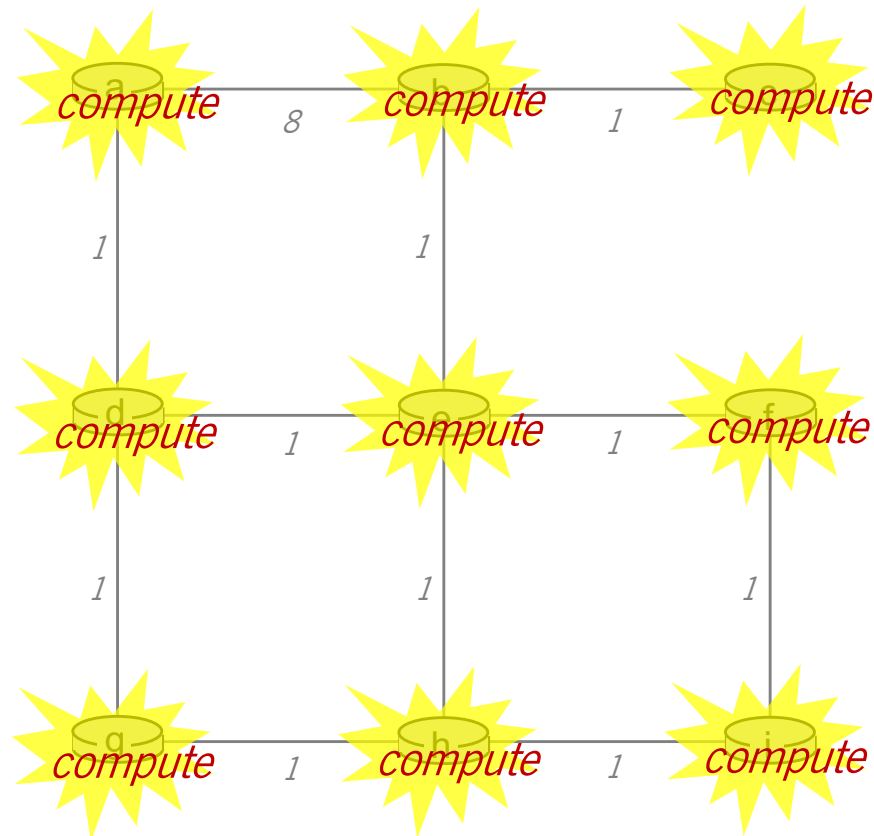
Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

All nodes:

- *receive distance vectors from neighbors*
- *compute their new local distance vector*
- *send their new local distance vector to neighbors*



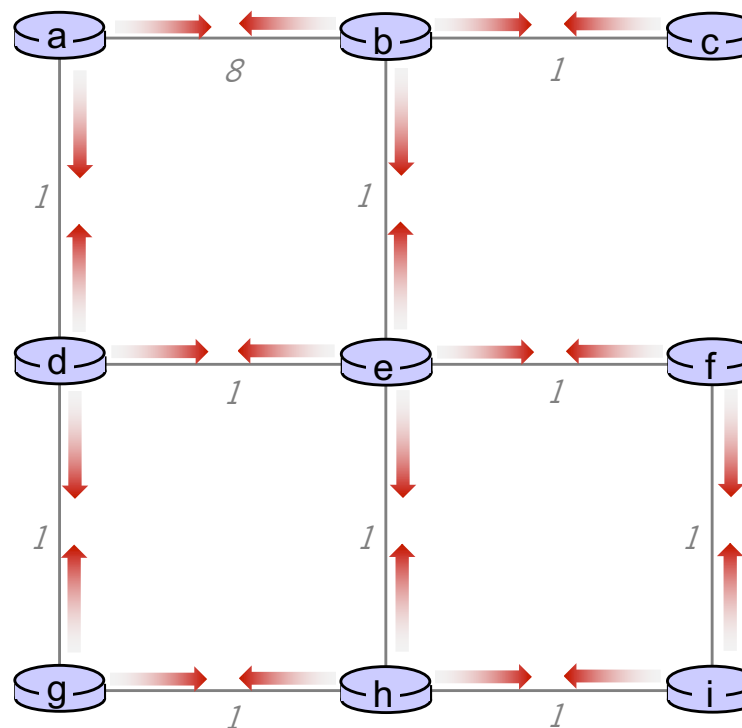
Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

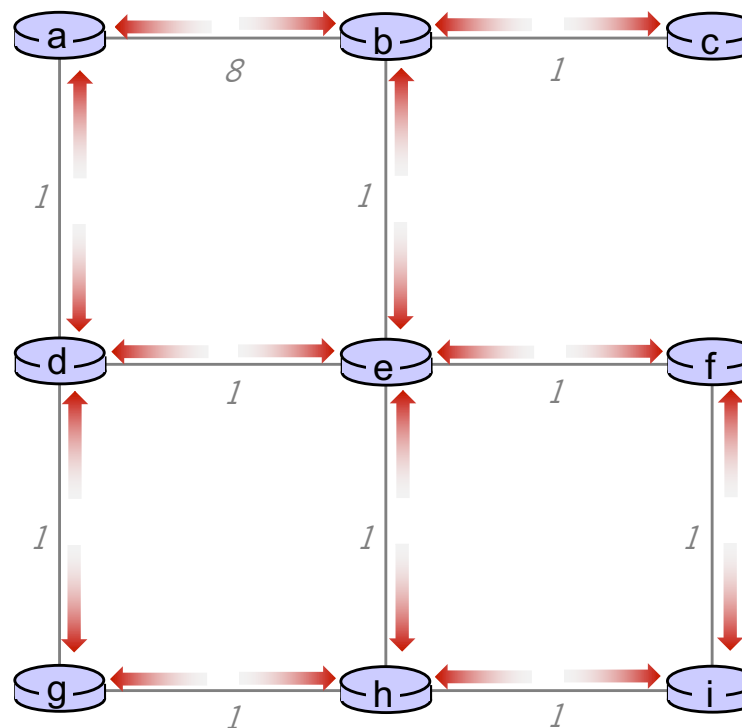


Algoritmo de Vetor de Distâncias (exemplo)

 $t=2$

All nodes:

- *receive distance vectors from neighbors*
- *compute their new local distance vector*
- *send their new local distance vector to neighbors*



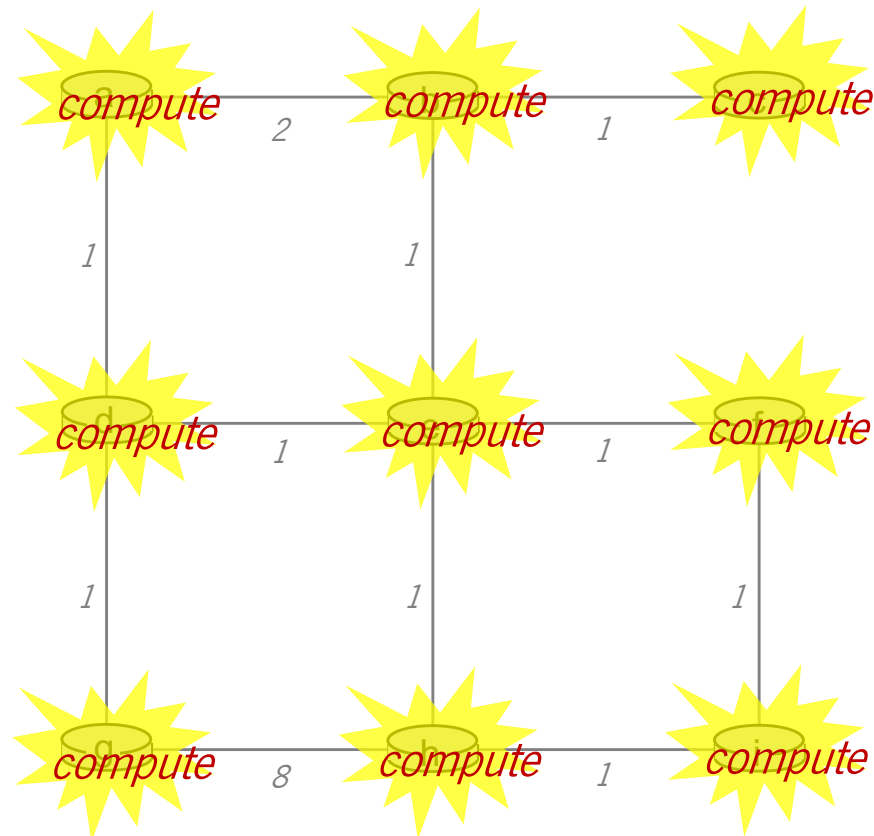
Algoritmo de Vetor de Distâncias (exemplo)



$t=2$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



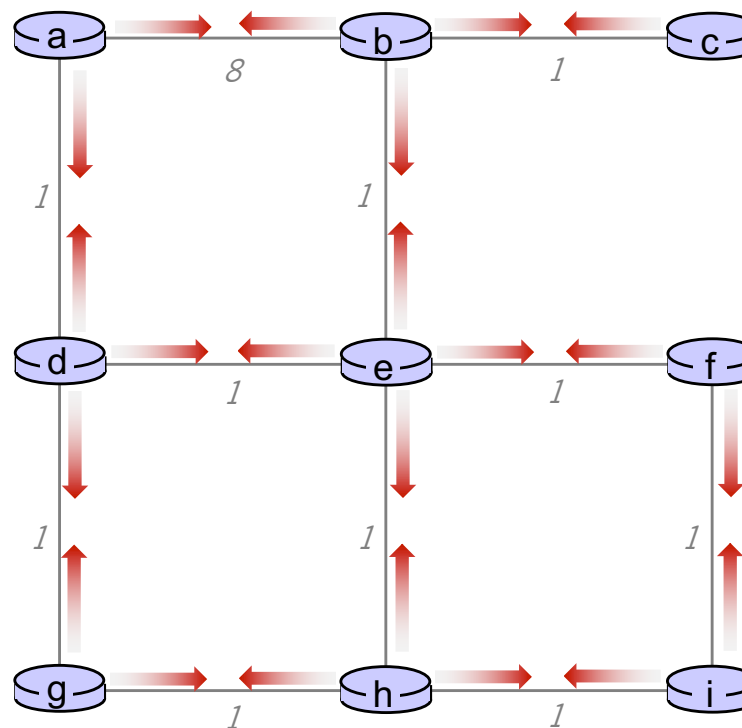
Algoritmo de Vetor de Distâncias (exemplo)



$t=2$

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



Algoritmo de Vetor de Distâncias (exemplo)



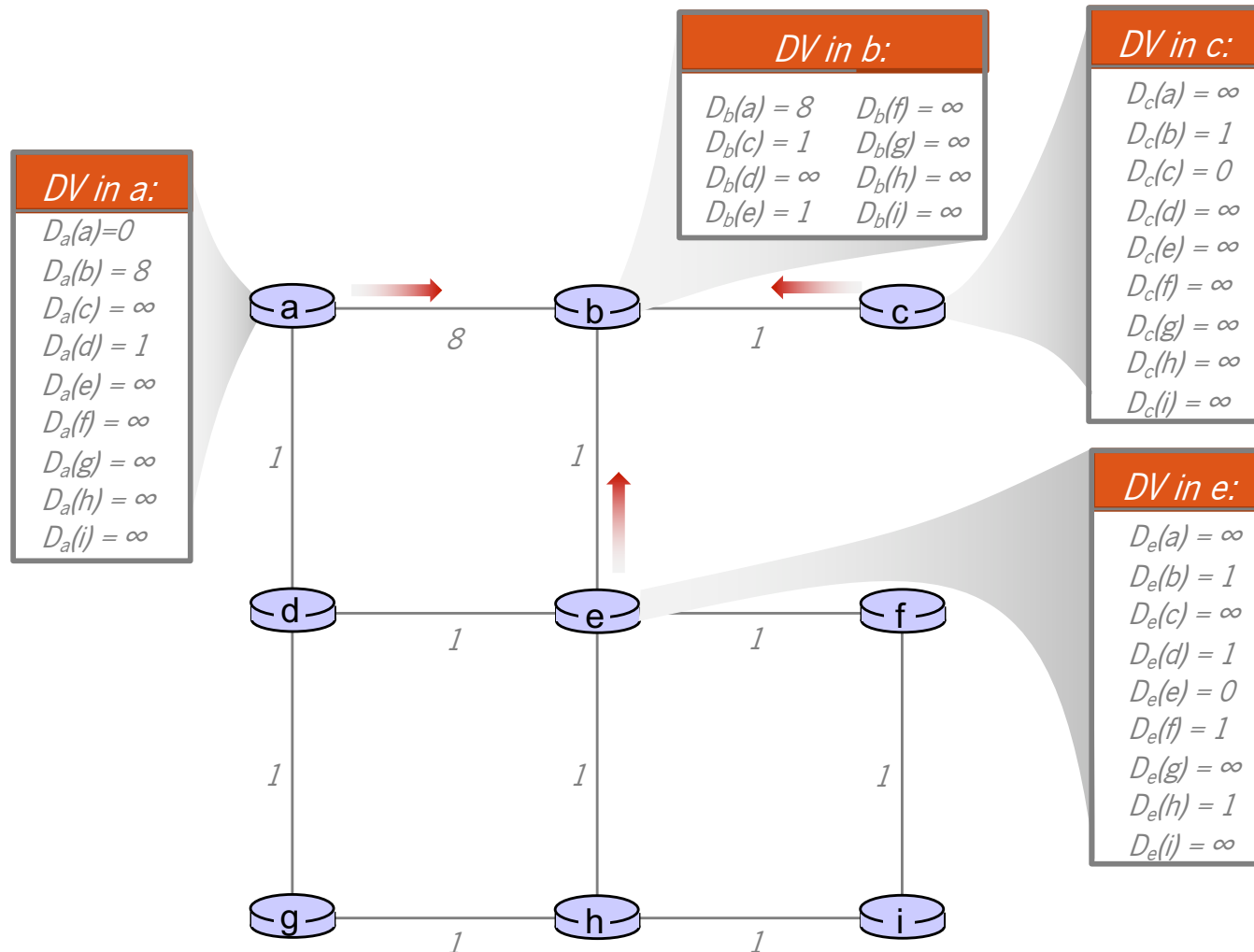
.... e assim sucessivamente

Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

- b receives DVs from a, c, e



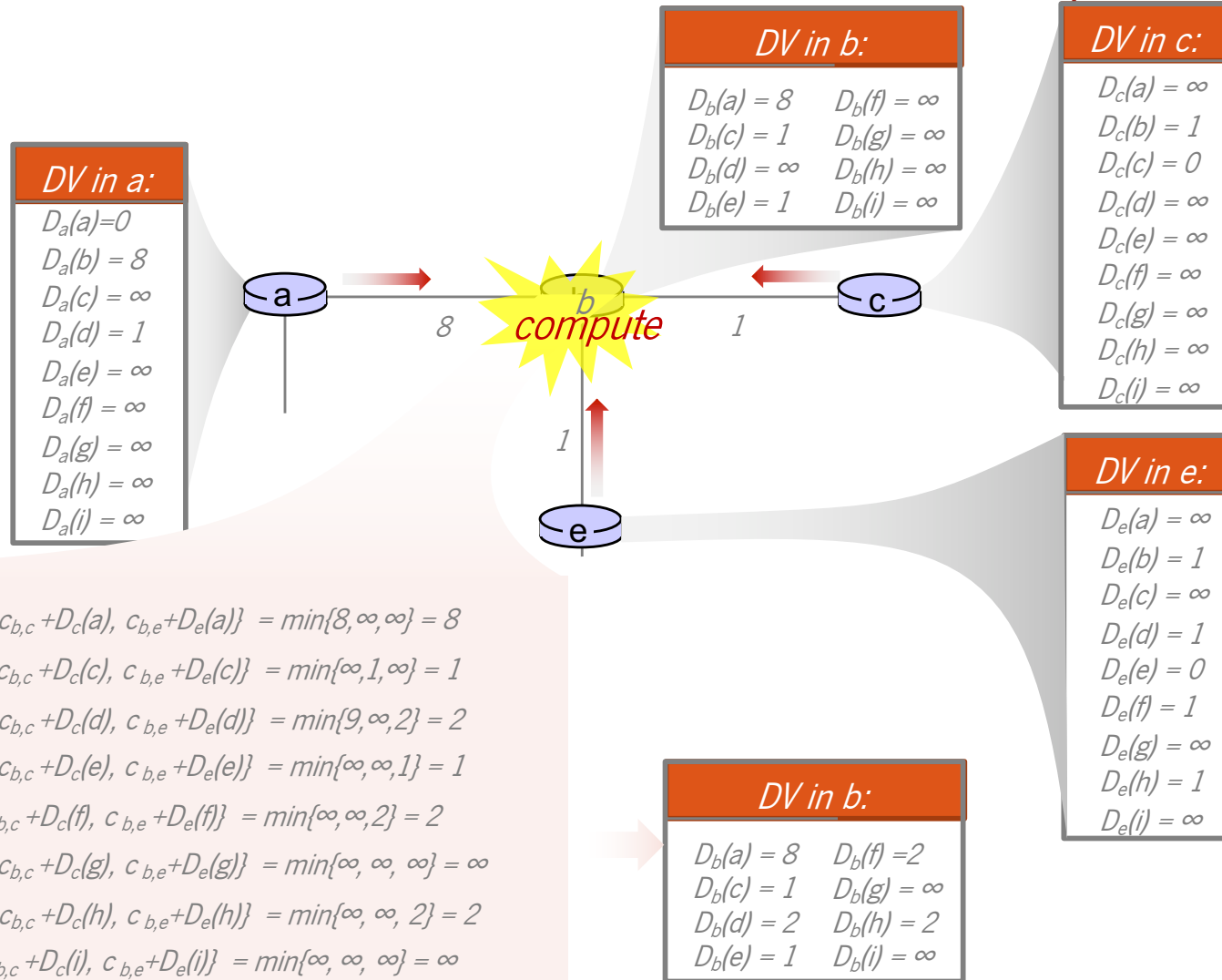
Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

- b receives DVs from a , c , e ,

$$\begin{aligned}
 D_b(a) &= \min\{c_{b,a} + D_a(a), c_{b,c} + D_c(a), c_{b,e} + D_e(a)\} = \min\{8, \infty, \infty\} = 8 \\
 D_b(c) &= \min\{c_{b,a} + D_a(c), c_{b,c} + D_c(c), c_{b,e} + D_e(c)\} = \min\{\infty, 1, \infty\} = 1 \\
 D_b(d) &= \min\{c_{b,a} + D_a(d), c_{b,c} + D_c(d), c_{b,e} + D_e(d)\} = \min\{9, \infty, 2\} = 2 \\
 D_b(e) &= \min\{c_{b,a} + D_a(e), c_{b,c} + D_c(e), c_{b,e} + D_e(e)\} = \min\{\infty, \infty, 1\} = 1 \\
 D_b(f) &= \min\{c_{b,a} + D_a(f), c_{b,c} + D_c(f), c_{b,e} + D_e(f)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(g) &= \min\{c_{b,a} + D_a(g), c_{b,c} + D_c(g), c_{b,e} + D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty \\
 D_b(h) &= \min\{c_{b,a} + D_a(h), c_{b,c} + D_c(h), c_{b,e} + D_e(h)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(i) &= \min\{c_{b,a} + D_a(i), c_{b,c} + D_c(i), c_{b,e} + D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty
 \end{aligned}$$



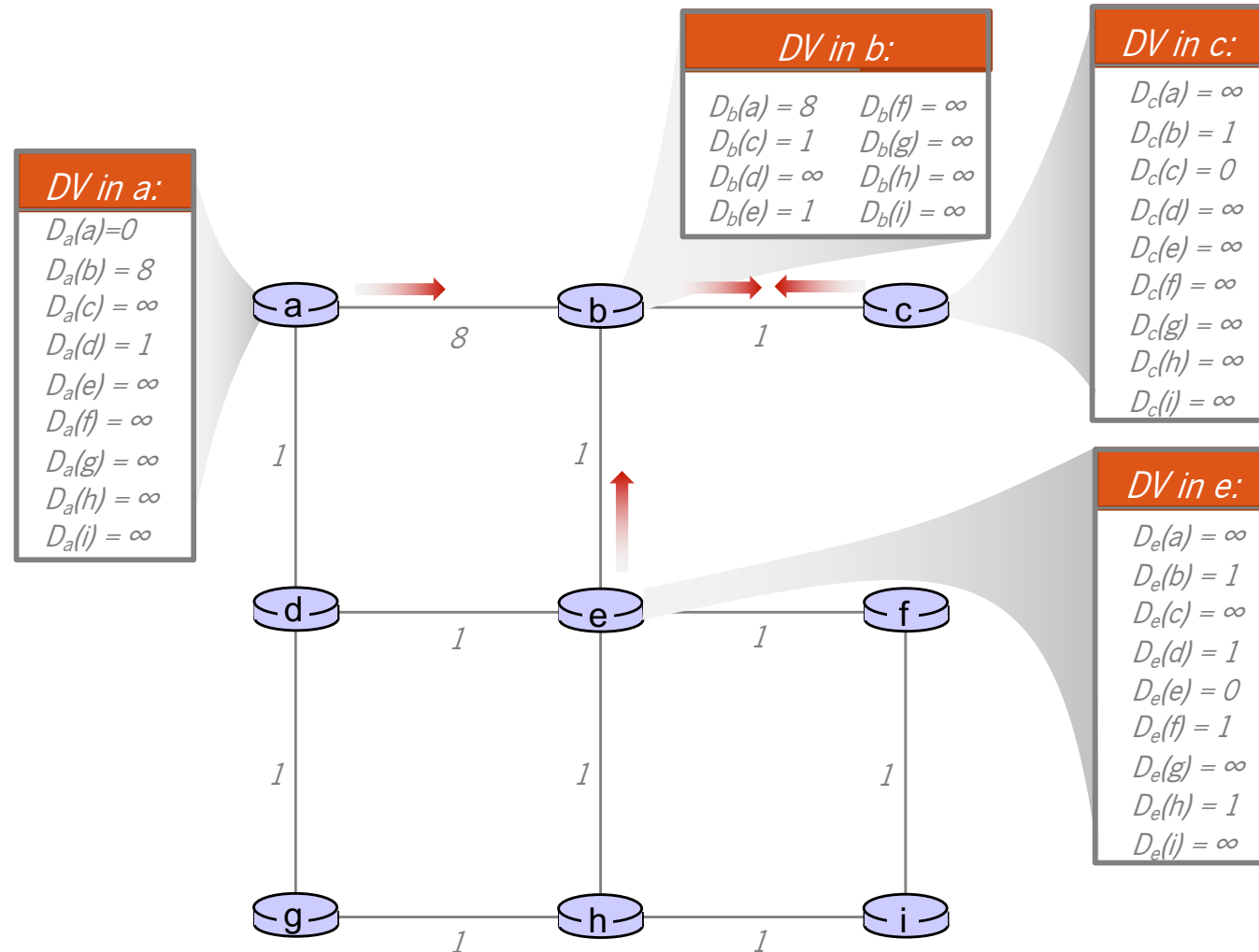


Algoritmo de Vetor de Distâncias (exemplo)

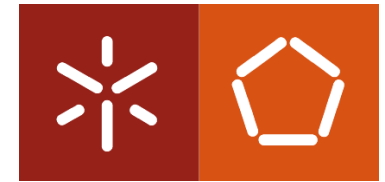


$t=1$

- c receives DVs from b



Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

- c receives DVs from b
computes:

$$D_c(a) = \min\{c_{c,b} + D_b(a)\} = 1 + 8 = 9$$

$$D_c(b) = \min\{c_{c,b} + D_b(b)\} = 1 + 0 = 1$$

$$D_c(d) = \min\{c_{c,b} + D_b(d)\} = 1 + \infty = \infty$$

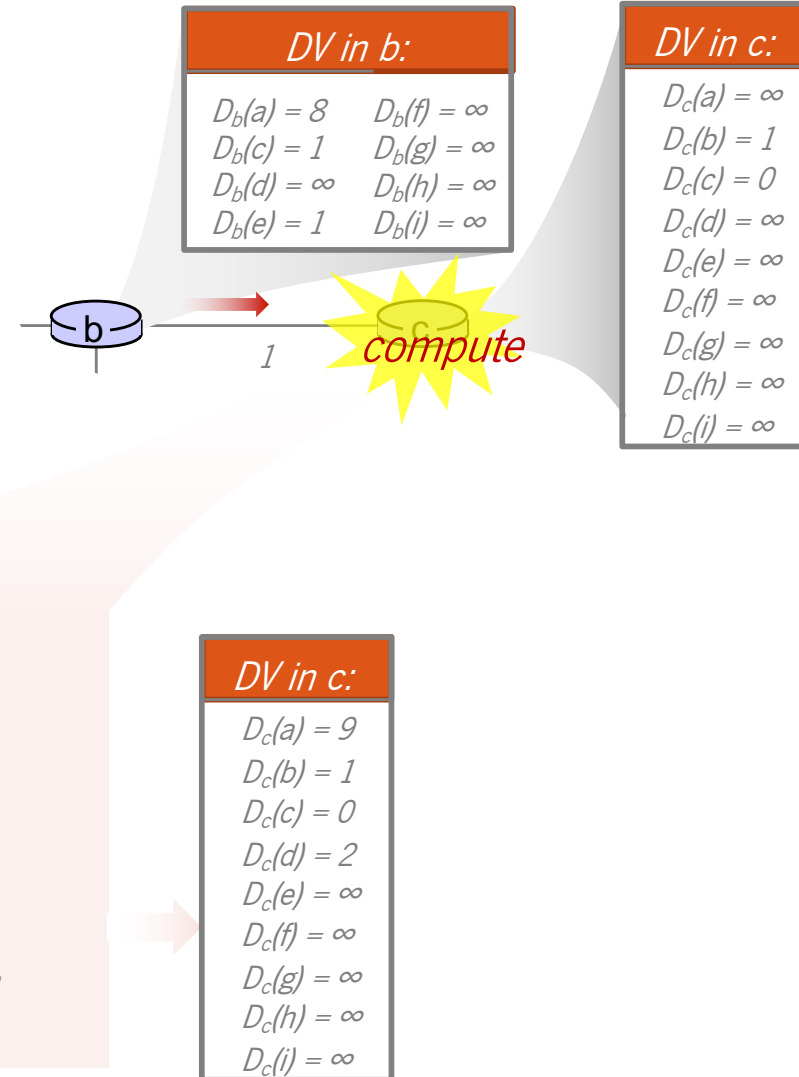
$$D_c(e) = \min\{c_{c,b} + D_b(e)\} = 1 + 1 = 2$$

$$D_c(f) = \min\{c_{c,b} + D_b(f)\} = 1 + \infty = \infty$$

$$D_c(g) = \min\{c_{c,b} + D_b(g)\} = 1 + \infty = \infty$$

$$D_c(h) = \min\{c_{c,b} + D_b(h)\} = 1 + \infty = \infty$$

$$D_c(i) = \min\{c_{c,b} + D_b(i)\} = 1 + \infty = \infty$$

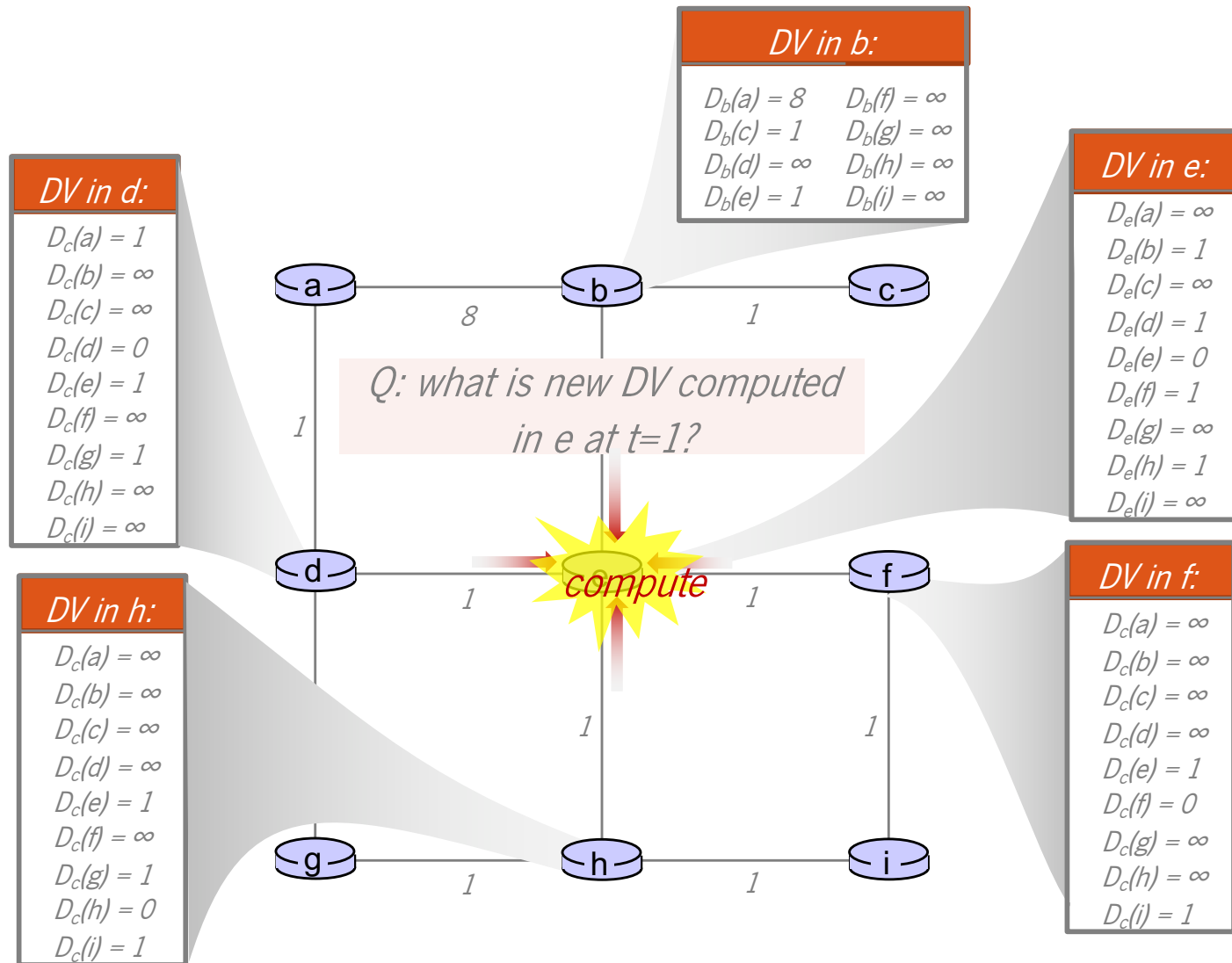


Algoritmo de Vetor de Distâncias (exemplo)



$t=1$

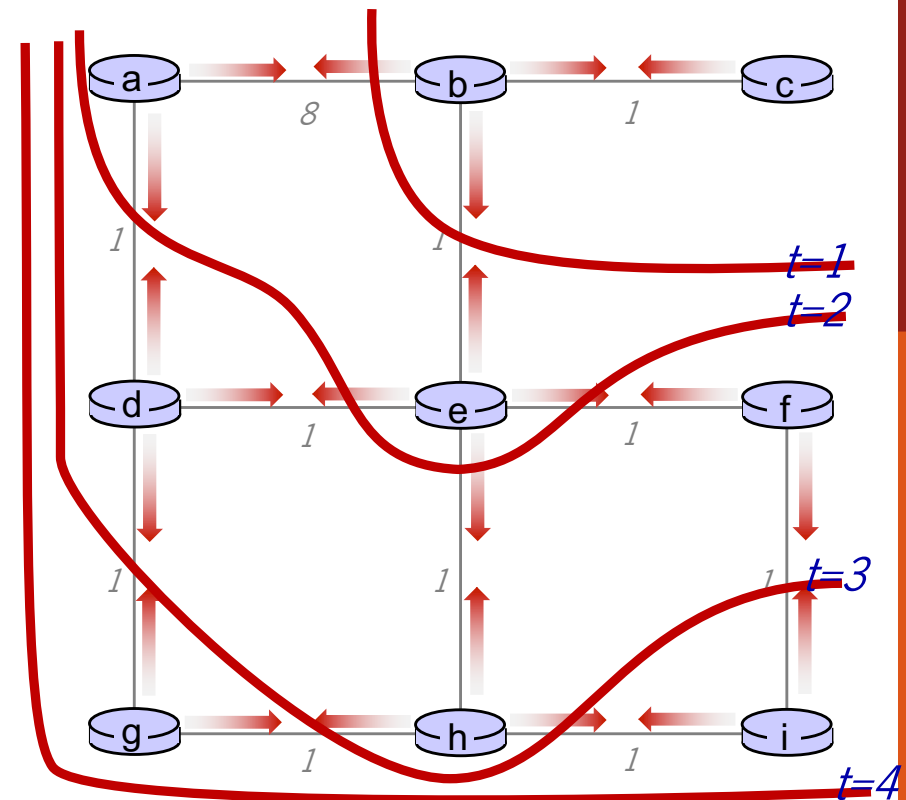
- e receives DVs from b, d, f, h



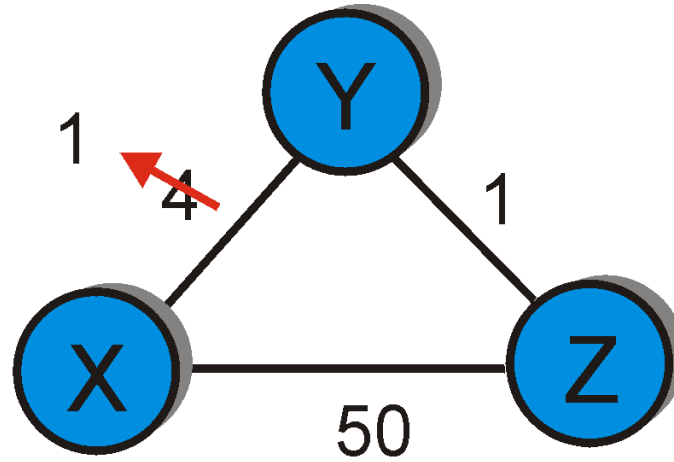
Vetor de Distâncias: difusão de informação de estado



-  $t=0$ c 's state at $t=0$ is at c only
-  $t=1$ c 's state at $t=0$ has propagated to b , and may influence distance vector computations up to **1** hop away, i.e., at b
-  $t=2$ c 's state at $t=0$ may now influence distance vector computations up to **2** hops away, i.e., at b and now at a, e as well
-  $t=3$ c 's state at $t=0$ may influence distance vector computations up to **3** hops away, i.e., at b, a, e and now at c, f, h as well
-  $t=4$ c 's state at $t=0$ may influence distance vector computations up to **4** hops away, i.e., at b, a, e, c, f, h and now at g, i as well



DVA: Good News...



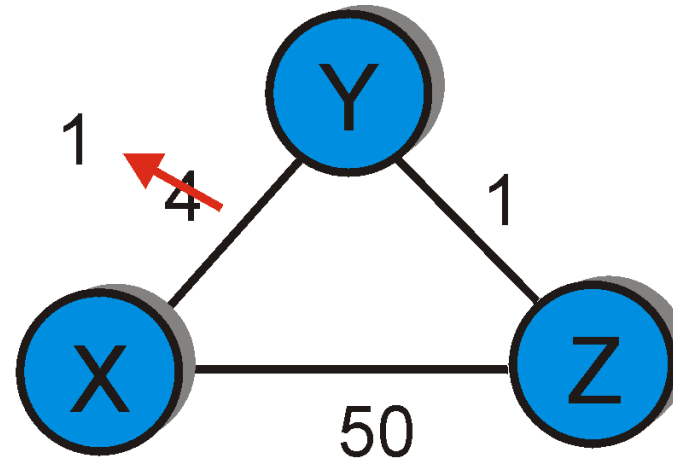
		via	
Y		X	Z
D			
x		4	6

		via	
Z		X	Y
D			
x		50	5

$c(X,Y)$
change



DVA: Good News... Travel Fast



D ^Y		via	
	X	Z	
x	(4)	6	

D ^Y			
	X	Z	
x	(1)	6	

D ^Y			
	X	Z	
x	(1)	6	

D ^Y			
	X	Z	
x	(1)	3	

D ^Z		via	
	X	Y	
x	50	(5)	

D ^Z			
	X	Y	
x	50	(5)	

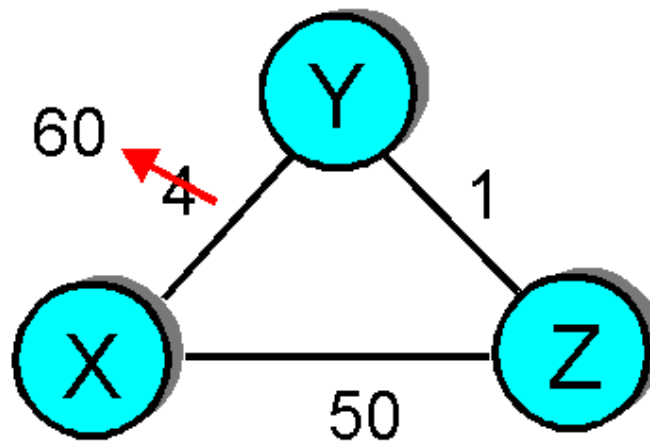
D ^Z			
	X	Y	
x	50	(2)	

D ^Z			
	X	Y	
x	50	(2)	

$c(X,Y)$
change



DVA: Bad News...

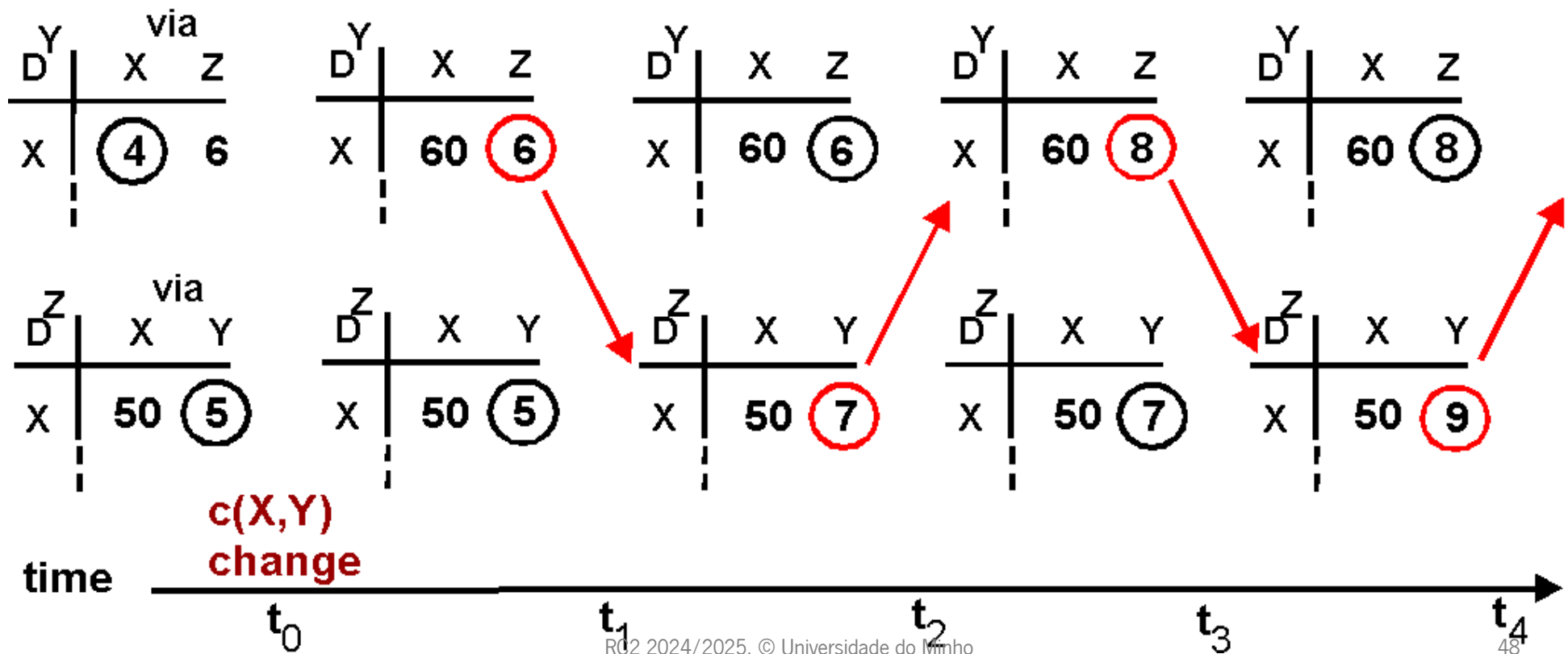
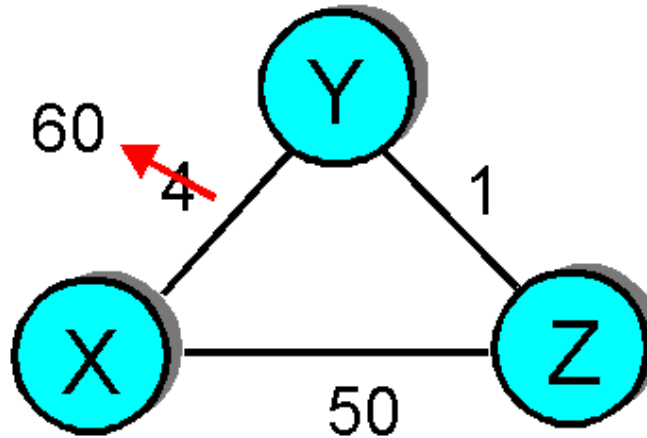


		via	
D ^Y		X	Z
X		4	6

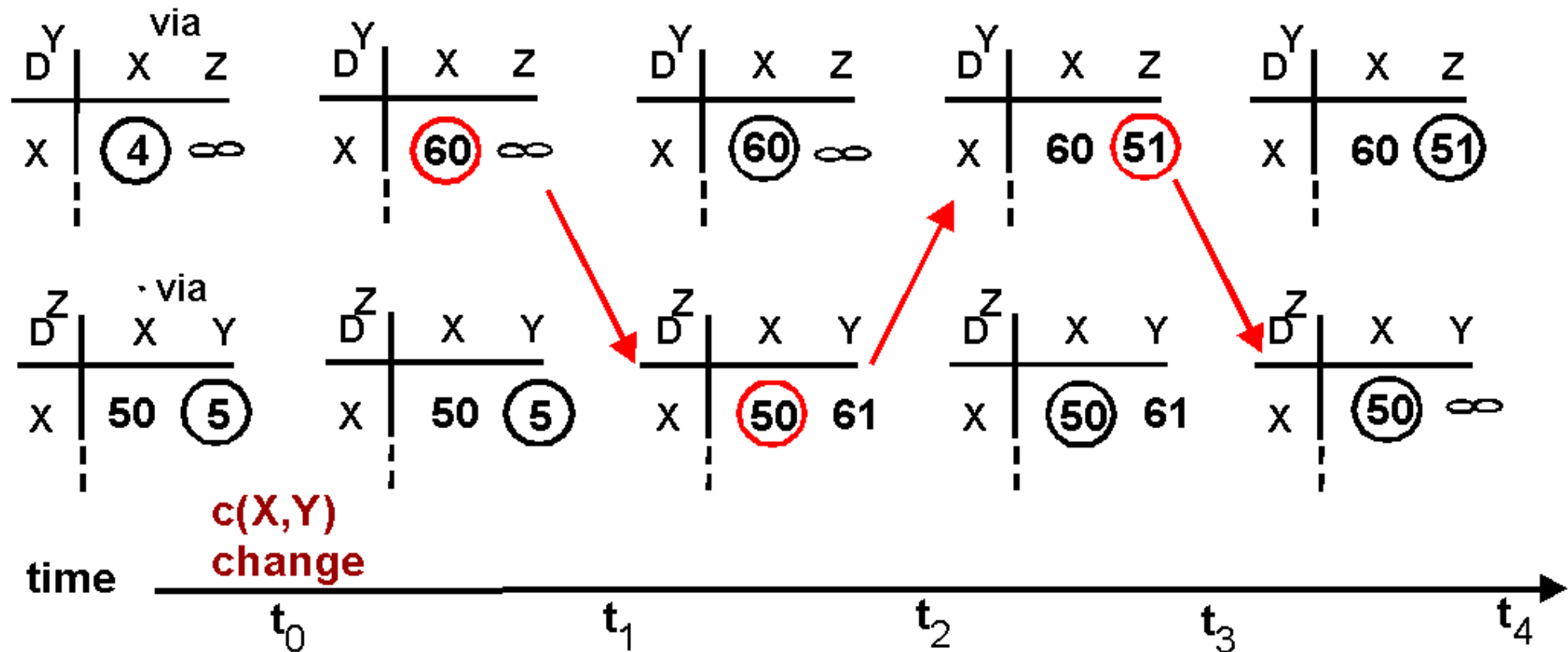
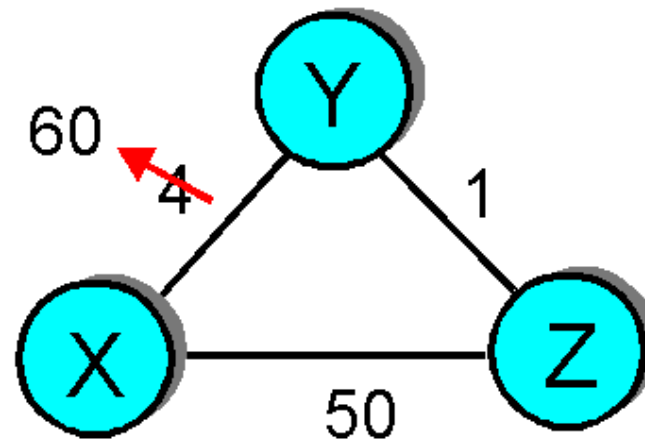
		via	
D ^Z		X	Y
X		50	5



DVA: Bad News... Count to Infinity!



DVA: Bad News... Count to Infinity!





● Sobrecarga introduzida pela mensagens de controle

- nos algoritmos de estado de ligação todos os nós necessitam de conhecer o custo de todas as ligações, por isso sempre que o custo de uma ligação muda, uma mensagem com o novo custo tem que ser enviada para todos os nós
- nos algoritmos de vector de distâncias a mudança do custo de uma ligação só provoca o envio de mensagens se resultar na mudança da tabela de encaminhamento

● Convergência

- os algoritmos de estado de ligação convergem mais depressa mas, com algumas métricas estão sujeitos a oscilações;
- em contrapartida os algoritmos de vector de distâncias convergem lentamente, podem apresentar ciclos enquanto não convergem e sofrem do problema da contagem até ao infinito.



● Robustez

- nos algoritmos de estado de ligação cada encaminhador calcula a sua tabela de encaminhamento usando a base de dados topológica, de forma independente dos outros encaminhadores. Isso confere a este tipo de algoritmos, robustez
- nos algoritmos de vector de distâncias se algum encaminhador estiver a calcular mal a sua tabela de encaminhamento, os erros cometidos vão-se propagar aos outros encaminhadores da topologia

● Recursos computacionais

- os algoritmos de estado de ligação são mais exigentes do que os algoritmos de vector de distâncias, quer em termos de memória (base de dados topológica versus tabela de distâncias) , quer em termos de capacidade de processamento